
Brian Vander Plaats
Developer
(https://stackoverflow.com/users/24892/brian-vander-plaats)
(https://www.linkedin.com/in/brianvanderplaats)
(https://github.com/brianplace)

Tree Data Structure Cheat Sheet

Tree Data Structure Cheat Sheet

References

Wikipedia Articles

- Binary Tree (https://en.wikipedia.org/wiki/Binary_tree)
- Binary Search Tree (BST) (https://en.wikipedia.org/wiki/Binary_search_tree)
- Tree Traversal (https://en.wikipedia.org/wiki/Tree_traversal)
- Tree Rotation (https://en.wikipedia.org/wiki/Tree_rotation)
- B-Tree (https://en.wikipedia.org/wiki/B-tree)
- AVL Tree (https://en.wikipedia.org/wiki/AVL_tree)
- Red-Black Tree (https://en.wikipedia.org/wiki/Red-black_tree)
- B+ Tree (https://en.wikipedia.org/wiki/B%2B_tree)

Definitions

- **Node** - an element containing data that make contain links to one or more parents/children May also be referred to as a vertex
- **Edge** - a connection between two nodes
- **Root** - The top node in a tree (a node without a parent)
- **Parent** - A node connected to another node when moving towards the root
- **Child** - a node connectd to another node when moving away from root
- **Descendant** - a node reachable by repeated processing from parent to child
- **Ancestor** - a node reachable by repeated processing from child to parent
- **Leaf** - a node without any children

- **Degree** - the number of sub trees of a node
- **Path** - a sequence of nodes and edges connecting a node with a descendant
- **Depth** - the depth of a node is the number of edges from the node to the tree's root node
- **Subtree** - of a tree T is a tree consisting of a node in T and all of it's descendants in T

Tree Types

Binary Tree

A binary tree is a tree data structure in which each node has at most two children.

- **Full Binary tree** - every node in the tree has 0 or 2 children
- **Perfect binary tree** - all interior nodes have two children and all leaves have the same depth or level
- **Complete binary tree** - every level except possibly the last is completely filled, and all nodes in the last level are as far left as possible

Binary Search Tree

A binary search tree (BST) is a data structure that binary tree that keeps it's keys in sorted order, so that operations can take advantage of the binary search principle (a logarithmic search that takes happens in O(log n) time)

B-tree

A B-tree is a self-balancing tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in logarithmic time. It is a generalization of a binary search tree in that a node can have more than two children. A B-tree is optimized for systems that read and write large blocks of data. B-tree's are commonly used in databases and file systems.

B+ Tree

A B+ tree is a B-tree in which each node only contains keys (not key-values), and to which an additional level is added at the bottom with linked leaves.
This makes for more efficient retrieval of data in block-oriented storage (once you find the start of the block, you can read sequentially without having to traverse up and down the tree to retrieve data nodes). Additionally, all leave nodes must be the same distance from the root node.

SQL Server & Oracle store table indexes in B+ trees, which are similar to B-trees, except that data is only stored in leaf nodes - all other nodes hold only key values and pointers to the next nodes.

AVL Tree

An AVL Tree is a self-balancing binary search tree. The height of the two child subtrees of any node differ at most by one, otherwise the tree is re-balanced. Lookup, insertion, and deletion take O(log n) time. Insertions and deletion may cause a tree rotation

Red-Black Tree

A red-black tree is a self-balancing binary search tree. Each node of the tree has an extra bit, which is interpreted as either black or red. The color bits are used to ensure the tree remains balanced during insertions and deletions. Operations occur in O(log n) time.

Tree Storage

Binary Tree

Pointer-based

Store references to parent and children

```
public class Node
{
        public Node Parent {get;set;}
        public Node Left {get;set;}
        public Node Right {get;set;}
    public int Key {get;set}
        public Object Data {get;set;}
}

public class BinaryTree
{
    protected Node Root {get;set}

    public BinaryTree()
    {
    }

    public void Insert(int key, Object data)
    {

    }

    public void Delete(int key)
    {

    }

    public Node Search(int key)
    {

    }
}
```

Array Based

if a node is at index i, left child is at index (2i + 1), right child is at (2i + 2).
Note that this will result in a lot of wasted space if the tree is not balanced! In other words, a complete binary tree is a good candidate for array-based storage

Tree Rotation

An operation on a binary tree that changes the structure without interfering with the order of the elements.

```
// Note these methods haven't been tested yet...

// Need to update up to three edges
// Edge #1 - Pivot -> It's parent
// Edge #2 - Pivot's parent -> Pivot's Grandparent (it exists)
// Edge #3 - Pivot's right child becomes left child of Pivot's parent
public void RotateRight(Node pivot)
{
    Node currentRoot = pivot.Parent;
    Node pivotRight = pivot.right;
    Node rootParent = currentRoot != null ? currentRoot.Parent : null;

    //Update root parent references
    if (rootParent != null )
    {
        rootParent.Left = rootParent.Left == currentRoot ? pivot : rootParent.Left;
        rootParent.Right = rootParent.Right == currentRoot ? pivot : rootParent.Right;
    }

    pivot.Parent = rootParent

    pivot.Right = currentRoot;
    currentRoot.Parent = pivot;

    //We don't know if the pivot is the right node or left node of the current root
    currentRoot.Right = currentRoot.Right == pivot ? pivotRight : currentRoot.Right;
    currentRoot.Left = currentRoot.Left == pivot ? pivotRight : currentRoot.Left;

}


// Need to update up to three edges
// Edge #1 - Pivot -> It's parent
// Edge #2 - Pivot's parent -> Pivot's Grandparent (it exists)
// Edge #3 - Pivot's left child becomes right child of Pivot's parent
public void RotateLeft(Node pivot)
{
    Node currentRoot = pivot.Parent;
    Node pivotLeft = pivot.Left;
    Node rootParent = currentRoot != null ? currentRoot.Parent : null;

    //Update root parent references
    if (rootParent != null )
    {
        rootParent.Left = rootParent.Left == currentRoot ? pivot : rootParent.Left;
        rootParent.Right = rootParent.Right == currentRoot ? pivot : rootParent.Right;
    }

    pivot.Parent = rootParent

    pivot.Left = currentRoot;
    currentRoot.Parent = pivot;

    //We don't know if the pivot is the right node or left node of the current root
    currentRoot.Right = currentRoot.Right == pivot ? pivotLeft : currentRoot.Right;
```

```
    currentRoot.Left = currentRoot.Left == pivot ? pivotLeft : currentRoot.Left;

}
```

Tree Traversal

Depth-first search

In a depth-first search, the search is deepened as much as possible on each child before going to the next sibling

Pre-Order

1. Display data of root
2. Traverse left subtree calling preorder function
3. Traverse right subtree calling preorder function

In-Order

1. Traverse left subtree calling preorder function
2. Display data of root
3. Traverse right subtree calling preorder function

An In-Order search will return the sorted contents of a BST (Binary Search Tree)

Post-Order

1. Traverse left subtree calling preorder function
2. Traverse right subtree calling preorder function
3. Display data of root

A stack can be used to perform a depth-first search

Breadth-first search

In a breadth-first search, all nodes on a level are visited before going to a lower level.

A Queue is often used to peform a breadth-first search

```
Queue nodes = new Queue();
StringBuilder OutputBuffer = new StringBuilder();
nodes.Enqueue(RootNode);

while (nodes.Count > 0 )
{
    Node node = (Node)nodes.Dequeue();

    OutputBuffer.Append(( OutputBuffer.Length > 0 ? "," : "")  + node.Value.ToString());

    if (node.LeftChild != null)
    {
        nodes.Enqueue(node.LeftChild);
    }
    if (node.RightChild != null)
    {
        nodes.Enqueue(node.RightChild);
    }
}
```

Searching a BST

Recursively

```
public Node Search(int key, Node node)
{
    if (node == null || Node.Key == key )
        return Node;
    else if (key < node.Key)
        return Search(key, node.Left);
    else // (key > node.Key)
        return Search(key, node.Right);
}
```

Iteratively

```
public Node Search(int key, Node node)
{
    Node current = node;
    while (current != null)
    {
        if (key == current.Key)
            return current;
        else if (key < current.Key)
            current = current.Left;
        else // (key > current.Key)
            current = current.Right;
    }

    return null; // didn't find the key :(
}
```

BST Insertions

```
// Recursive
public void Insert(Node root, int key, Object data)
{
    if (this.Root == null)
    {
        this.Root = new Node(key, data);
        return;
    }

    //set current subtree root to tree's root
    if (root == null )
        root = this.Root;

    if (key <= root.Key)
    {
        if (root.Left == null)
            root.Left = new Node(key, data);
        else
            Insert(root.Left, key, data);
    }
    else // key > root.Key
    {
        if (root.Right == null)
            root.Right = new node(key, data);
        else
            Insert(root.Right, key, data);
    }
}

// Interative
public void Insert(Node root, int key, Object data)
{
    if (this.Root == null)
    {
        this.Root = new Node(key, data);
        return;
    }

    Node current = this.Root;

    while (current != null )
    {
        if (key <= current.Key)
        {
            if (current.Left == null)
            {
                current.Left = new Node(key,data);
                current = null;
            }
            else
                current = current.Left;
        }
        else // key > current.Key
```

```
        {
            if (current.Right == null)
            {
                current.Right = new Node(key,data);
                current = null;
            }
            else
                current = current.Right
        }
    }

}
```